

Universidade Federal de Minas Gerais

Escola de Engenharia

Departamento de Engenharia Eletrônica

**Controle de Manipulador Robótico via Integração com
Servidor OPC UA**

Ana Paula Amorim Almeida Matrícula: 2021070888
Cássio Antônio Magalhães de Souza Almeida Matrícula:
2020023525

Julho de 2025

Conteúdo

1	Introdução	2
2	Descrição do Problema	2
3	Arquitetura do Sistema	3
3.1	CLP	3
3.2	Cliente TCP	5
3.3	MES	5
4	Fluxo de Operação	5
5	Testes Realizados	6
6	Conclusão	9

1 Introdução

Este relatório descreve o desenvolvimento de um sistema de controle para um manipulador robótico simulado no CoppeliaSim, integrando comunicação industrial via servidor OPC UA. O objetivo é consolidar os conceitos de comunicação industrial, supervisão e controle distribuído.

2 Descrição do Problema

Robôs manipuladores, como o Franka Emika Panda, são amplamente utilizados na indústria para tarefas que exigem precisão e repetibilidade. A integração com sistemas supervisórios, CLPs e MES exige uma comunicação confiável e padronizada, papel desempenhado pelo protocolo OPC UA.

Este trabalho tem como objetivo mover o manipulador a partir de um comando enviado pelo cliente TCP e mostrar no terminal as novas coordenadas X, Y e Z da garra do manipulador. Somado a isso, as operações realizadas e as posições devem ser salvas em dois arquivos .txt. O robô a ser controlador pode ser observado na Figura 1.



Figura 1: Manipulador robótico a ser controlado.

A estrutura da rede distribuída, englobando o caminho do cliente TCPI até o manipulador no CoppeliaSim, somado com o sistema MES, pode ser observado na Figura 2.

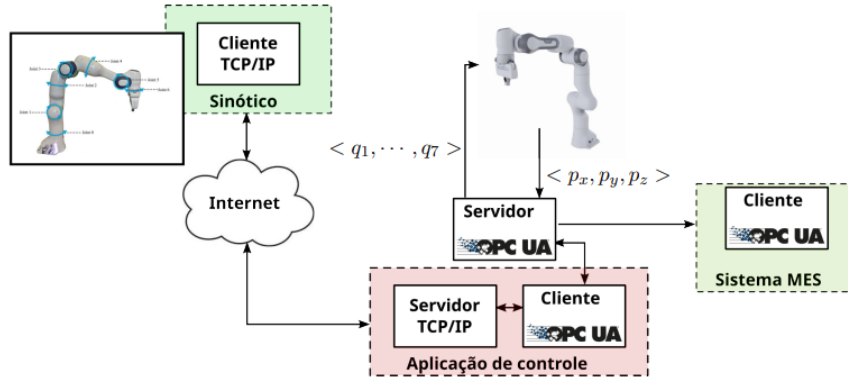


Figura 2: Sistema Distribuído para a operação do processo.

As funções e os programas em que estão contidos é explicado na próxima seção.

3 Arquitetura do Sistema

O sistema foi dividido em quatro módulos principais e os programas foram feitos na linguagem python:

- **Bridge:** Faz a ponte entre o servidor OPC UA e o simulador CoppeliaSim. Lê valores das juntas do servidor OPC UA e envia os comandos para o robô simulado.
- **CLP:** Cliente OPC UA responsável por ler e atuar no robô via servidor e um servidor TCP/IP para receber comandos do operador.
- **Cliente Supervisório (TCP/IP):** Permite ao operador enviar comandos para mover as juntas e receber o feedback de posição, registrando todas as operações no arquivo `historiador.txt`.
- **MES:** Cliente OPC UA que realiza a leitura periódica da posição da garra e registra os dados em `mes.txt`.

3.1 CLP

O programa `clp.py` possui três threads, uma para o Servidor TCP/IP, uma para o Cliente OPC UA e uma thread responsável para monitorar o terminal caso o usuário deseje sair e finalizar o programa.

Na Figura 3 pode ser observado parte do código referente ao Servidor TCP/IP.

```

73 def tcp_server():
74
75     global juntas_opc, px_node, py_node, pz_node, opc_client, idx, val_deg, resposta, comando
76
77     print_warn("[TCP] Servidor TCP ouvindo...")
78     HOST = '127.0.0.1'
79     PORT = 5858
80
81     with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
82         s.bind((HOST, PORT))
83         s.listen(1)
84         print_ok(f"[TCP] Servidor TCP/IP ativo em {HOST}:{PORT}")
85
86         while True:
87             conn, addr = s.accept()
88             print_warn(f"[TCP] Conexão recebida de {addr}")
89             with conn:
90                 while True:
91                     data = conn.recv(1024)
92                     if not data:
93                         break
94
95                     try:
96                         msg = data.decode().strip()
97                         print(f"[TCP] Comando recebido: {msg}")
98                         junta_str, angulo_str = msg.split(",")
99
100                         with mutex:
101                             idx = int(junta_str)
102                             val_deg = float(angulo_str)
103                             comando = True
104
105                         if idx < 3:
106                             time.sleep(10) # Pra dar tempo de as posicoes serem alteradas, as vezes demora menos, mas nao encontrei um padrao
107                         else:
108                             time.sleep(15) # Pra dar tempo de as posicoes serem alteradas
109
110                         with mutex:
111                             if resposta:
112                                 conn.sendall(resposta.encode())
113                             else:
114                                 conn.sendall(b"Erro: Resposta ainda nao gerada\n")
115
116                     except Exception as e:
117                         conn.sendall(f"Erro: {str(e)}\n".encode())
118                         print(f"[TCP] Encerrando thread do servidor")
119

```

Figura 3: Código da Thread do Servidor TCP/IP.

Após o estabelecimento da conexão com o cliente TCP, a thread entra em um loop, onde separa a mensagem recebida em partes e, após obter o mutex, altera as variáveis "idx", "val_deg" e "comando", responsáveis por, respectivamente, indicar a junta que deve mexer, o valor desejado e a presença de um novo comando a ser realizado.

Somado a isso, após devolver o mutex, foi realizado um comando de sleep (linhas 105 à 108). Isso foi necessário pois, mesmo após a alteração do valor das juntas no Prosys OPC UA, o CoppeliaSim apresentava um delay para movimentar o manipulador. Esse delay não era fixo, então colocamos no código o maior delay observado, pois, caso contrário, poderia ocorrer uma leitura das posições X, Y e Z incorretas. Desconfiando que poderia ser um problema no uso de mutex, fizemos um código mais simples para testar, mas o delay permaneceu, podendo estar relacionado com a máquina em que o código foi executado.

ATENÇÃO: É importante ressaltar que, caso a máquina do usuário não apresente esse delay no CoppeliaSim, basta comentar essas linhas do código para tornar sua execução mais rápida e dinâmica.

Para finalizar o loop, a thread aguarda o mutex para enviar a mensagem que está escrita na variável "resposta", atualizada pela thread do cliente OPC.

Na Figura 4 pode ser observado parte do código referente ao Cliente OPC UA.

```

def connect_opcua():
    global juntas_opc, px_node, py_node, pz_node, opc_client, idx, val_deg, resposta, comando

    try:
        print_warn("[OPC] Iniciando cliente OPC UA...")
        opc_client = Client("opc.tcp://localhost:53530/OPCUA/SimulationServer")
        opc_client.connect()
        print_ok("[OPC] Conectado ao servidor OPC UA")

        root = opc_client.get_objects_node()
        franka = root.get_child(["3:Franka"])
        filhos = franka.get_children()

        if len(filhos) < 10:
            raise Exception("Variaveis OPC UA incompletas")

        juntas_opc = filhos[:7]
        px_node, py_node, pz_node = filhos[-3:]

        print_ok("[OPC] Variaveis das juntas e posicao carregadas com sucesso")

        while True:
            with mutex:
                if comando:
                    juntas_opc[idx-1].set_value(val_deg)
                    print(f"[OPC] Junta {idx} ajustada para {val_deg:.2f} graus")
                    comando = False

            # Le a posicao da garra
            px = px_node.get_value()
            py = py_node.get_value()
            pz = pz_node.get_value()
            timestamp = datetime.now().strftime('%d-%m-%Y %H:%M:%S')

            with mutex:
                resposta = f"{px:.3f},{py:.3f},{pz:.3f},{timestamp}"

```

Figura 4: Código da Thread do Cliente OPC UA.

Após iniciar o cliente OPC, conectar-se ao servidor e linkar as variáveis relacionadas às juntas e posições a thread entra em um loop. Após pegar o mutex, ocorre a verificação se há algum comando pendente, em caso positivo, o programa irá alterar, no servidor OPC UA, a junta correspondente para o valor desejado. Em seguida, irá devolver o mutex e realizar a leitura das posições com seu timestamp, para, assim, atualizar a variável "resposta". Vale ressaltar que quando "comando = FALSE", a thread do cliente OPC irá apenas realizar a leitura das posições e atualizar "resposta".

Na Figura 5 pode ser observado parte do código referente à monitoração do terminal.

```

def monitorar_saida():
    while True:
        comando = input()
        if comando.lower() == "sair":
            print_warn("[CLP] Encerrando programa por comando do usuario.")
            os._exit(0) # Encerra imediatamente todas as threads

```

Figura 5: Código da Thread que monitora o terminal.

A única finalidade dela é encerrar as threads e o programa quando o usuário digitar "sair" no terminal.

3.2 Cliente TCP

3.3 MES

4 Fluxo de Operação

1. O servidor OPC UA (Prosys) é configurado com variáveis representando as juntas e posição da garra.
2. O CoppeliaSim é iniciado com o modelo do robô Franka Emika Panda.

3. O `bridge.py` conecta o servidor OPC UA com o simulador, enviando e recebendo dados.
4. O `clp.py` conecta ao servidor OPC UA e recebe comandos do `clienteTCP.py` via TCP/IP.
5. O `clienteTCP.py` permite ao operador controlar as juntas, registrar operações e visualizar feedback em tempo real.
6. O `mes.py` armazena os dados de posição da garra continuamente.

5 Testes Realizados

Os testes incluíram:

- Verificação de comunicação entre servidor OPC UA e CoppeliaSim.
- Movimentação individual de juntas por comandos TCP/IP.
- Registro de histórico de comandos no arquivo `historiador.txt`.
- Validação de leitura periódica pelo MES.

Para não estender o relatório iremos comentar apenas sobre um dos testes realizados. Após executar os programas "`bridge.py`", "`clp.py`", "`clienteTCP.py`" e "`mes.py`" (nessa ordem), foram inseridos diferentes comandos para diferentes juntas e os resultados podem ser observados nas figuras abaixo.

```
[CLIENTE] Conectado ao servidor.
Digite a junta (1 a 7, ou 'sair' para encerrar): 1
Digite o angulo desejado (entre -180 e 180): -90
[RESPOSTA] Posicao da garra: Px=0.098, Py=0.475, Pz=0.215 | Timestamp: 08-07-2025 15:30:05
Digite a junta (1 a 7, ou 'sair' para encerrar): 3
Digite o angulo desejado (entre -180 e 180): -40
[RESPOSTA] Posicao da garra: Px=-1.299, Py=0.475, Pz=0.444 | Timestamp: 08-07-2025 15:30:41
Digite a junta (1 a 7, ou 'sair' para encerrar): 2
Digite o angulo desejado (entre -180 e 180): 90
[RESPOSTA] Posicao da garra: Px=-1.208, Py=0.475, Pz=0.663 | Timestamp: 08-07-2025 15:30:59
Digite a junta (1 a 7, ou 'sair' para encerrar): 1
Digite o angulo desejado (entre -180 e 180): 0
[RESPOSTA] Posicao da garra: Px=-1.206, Py=0.808, Pz=0.329 | Timestamp: 08-07-2025 15:31:14
Digite a junta (1 a 7, ou 'sair' para encerrar): 7
Digite o angulo desejado (entre -180 e 180): 90
[RESPOSTA] Posicao da garra: Px=-0.600, Py=0.809, Pz=0.939 | Timestamp: 08-07-2025 15:31:42
Digite a junta (1 a 7, ou 'sair' para encerrar): 6
Digite o angulo desejado (entre -180 e 180): 90
[RESPOSTA] Posicao da garra: Px=-0.600, Py=0.809, Pz=0.939 | Timestamp: 08-07-2025 15:32:04
Digite a junta (1 a 7, ou 'sair' para encerrar): 5
Digite o angulo desejado (entre -180 e 180): 90
[RESPOSTA] Posicao da garra: Px=-0.600, Py=0.798, Pz=1.063 | Timestamp: 08-07-2025 15:32:29
Digite a junta (1 a 7, ou 'sair' para encerrar):
```

Figura 6: Terminal do Cliente TCP após série de comandos.

```

[TCP] Servidor TCP ouvindo...
[OPC] Iniciando cliente OPC UA...
[TCP] Servidor TCP/IP ativo em 127.0.0.1:5050
[OPC] Conectado ao servidor OPC UA
[OPC] Variaveis das juntas e posicao carregadas com sucesso
[TCP] Conexao recebida de ('127.0.0.1', 62368)
[TCP] Comando recebido: 1,-90.0
[OPC] Junta 1 ajustada para -90.00 graus
[TCP] Comando recebido: 3,-40.0
[OPC] Junta 3 ajustada para -40.00 graus
[TCP] Comando recebido: 2,90.0
[OPC] Junta 2 ajustada para 90.00 graus
[TCP] Comando recebido: 1,0.0
[OPC] Junta 1 ajustada para 0.00 graus
[TCP] Comando recebido: 7,90.0
[OPC] Junta 7 ajustada para 90.00 graus
[TCP] Comando recebido: 6,90.0
[OPC] Junta 6 ajustada para 90.00 graus
[TCP] Comando recebido: 5,90.0
[OPC] Junta 5 ajustada para 90.00 graus

```

Figura 7: Terminal do CLP após série de comandos.

```

[MES] Iniciando cliente OPC UA...
[MES] Conectado ao servidor OPC UA
[MES] Variaveis da garra carregadas com sucesso
[MES] 2025-07-08 15:29:40 - Px: 0.098, Py: 0.475, Pz: 0.215
[MES] 2025-07-08 15:29:55 - Px: 0.098, Py: 0.475, Pz: 0.215
[MES] 2025-07-08 15:30:10 - Px: 0.098, Py: 0.475, Pz: 0.215
[MES] 2025-07-08 15:30:25 - Px: -1.299, Py: 0.475, Pz: 0.444
[MES] 2025-07-08 15:30:40 - Px: -1.299, Py: 0.475, Pz: 0.444
[MES] 2025-07-08 15:30:55 - Px: -1.208, Py: 0.475, Pz: 0.663
[MES] 2025-07-08 15:31:10 - Px: -1.206, Py: 0.808, Pz: 0.329
[MES] 2025-07-08 15:31:25 - Px: -0.600, Py: 0.809, Pz: 0.939
[MES] 2025-07-08 15:31:40 - Px: -0.600, Py: 0.809, Pz: 0.939
[MES] 2025-07-08 15:31:55 - Px: -0.600, Py: 0.809, Pz: 0.939
[MES] 2025-07-08 15:32:10 - Px: -0.600, Py: 0.809, Pz: 0.939
[MES] 2025-07-08 15:32:25 - Px: -0.600, Py: 0.809, Pz: 0.939
[MES] 2025-07-08 15:32:40 - Px: -0.600, Py: 0.798, Pz: 1.063
[MES] 2025-07-08 15:32:55 - Px: -0.600, Py: 0.798, Pz: 1.063
[MES] 2025-07-08 15:33:10 - Px: -0.600, Py: 0.798, Pz: 1.063
[MES] 2025-07-08 15:33:25 - Px: -0.600, Py: 0.798, Pz: 1.063
[MES] 2025-07-08 15:33:40 - Px: -0.600, Py: 0.798, Pz: 1.063
[MES] 2025-07-08 15:33:55 - Px: -0.600, Py: 0.798, Pz: 1.063
[MES] 2025-07-08 15:34:10 - Px: -0.600, Py: 0.798, Pz: 1.063
[MES] 2025-07-08 15:34:25 - Px: -0.600, Py: 0.798, Pz: 1.063
[MES] 2025-07-08 15:34:40 - Px: -0.600, Py: 0.798, Pz: 1.063
[MES] 2025-07-08 15:34:55 - Px: -0.600, Py: 0.798, Pz: 1.063
[MES] 2025-07-08 15:35:10 - Px: -0.600, Py: 0.798, Pz: 1.063

```

Figura 8: Terminal do MES após série de comandos.

Na Figura 6, que após realizae o comando ele recebe as novas posições X, Y e Z da garra do manipulador. Ademais, na Figura 8, observa-se que as posições do manipulador são printadas periodicamente. A configuração final do manipulador pode ser observado na Figura 9

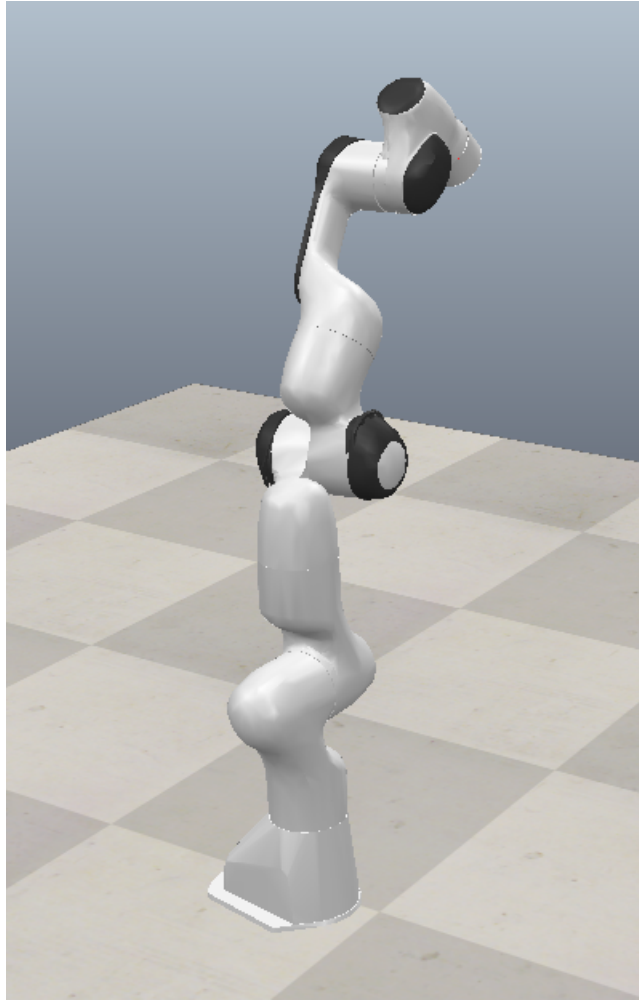


Figura 9: Configuração Final do manipulador após série de comandos.

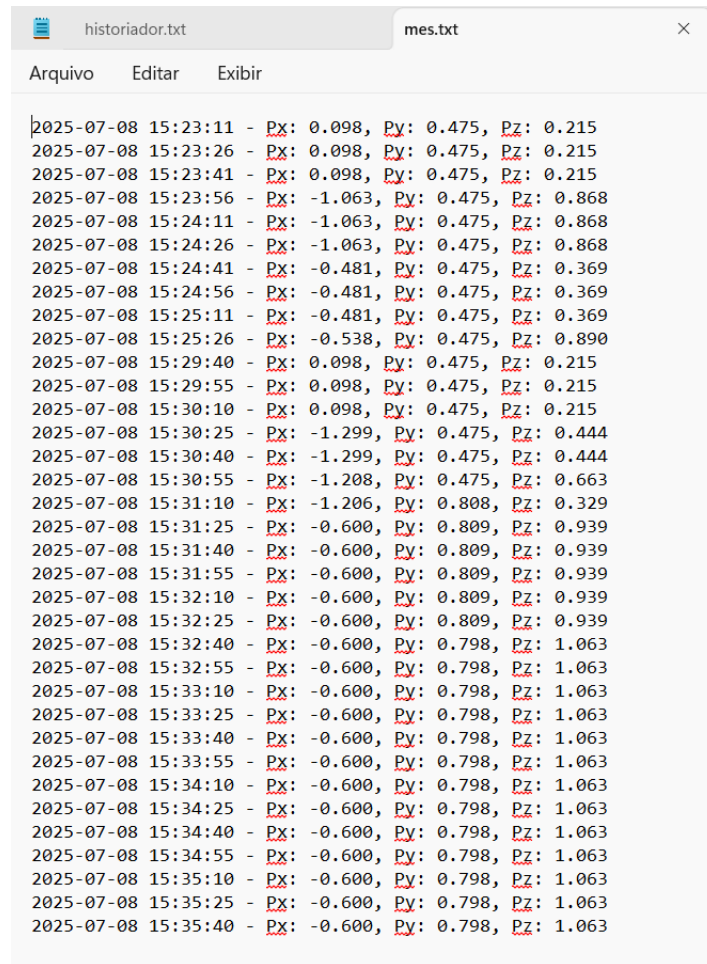
Para finalizar o teste basta verificar se os arquivos "historiador.txt" e "mes.txt" foram gerados e escritos corretamente.

```

historiador.txt  x  mes.txt
Arquivo  Editar  Exibir  H1  ≡
Junta: 1, Angulo: -50.0 | Posicao: (0.098, 0.475, 0.215) | Timestamp: 08-07-2025 15:23:45
Junta: 3, Angulo: 40.0 | Posicao: (-1.063, 0.475, 0.868) | Timestamp: 08-07-2025 15:24:09
Junta: 3, Angulo: -180.0 | Posicao: (-0.481, 0.475, 0.369) | Timestamp: 08-07-2025 15:24:37
Junta: 3, Angulo: -90.0 | Posicao: (-0.538, 0.475, 0.890) | Timestamp: 08-07-2025 15:25:16
Junta: 2, Angulo: 80.0 | Posicao: (-0.538, 0.475, 0.890) | Timestamp: 08-07-2025 15:25:33
Junta: 1, Angulo: -90.0 | Posicao: (0.098, 0.475, 0.215) | Timestamp: 08-07-2025 15:30:05
Junta: 3, Angulo: -40.0 | Posicao: (-1.299, 0.475, 0.444) | Timestamp: 08-07-2025 15:30:41
Junta: 2, Angulo: 90.0 | Posicao: (-1.208, 0.475, 0.663) | Timestamp: 08-07-2025 15:30:59
Junta: 1, Angulo: 0.0 | Posicao: (-1.206, 0.808, 0.329) | Timestamp: 08-07-2025 15:31:14
Junta: 7, Angulo: 90.0 | Posicao: (-0.600, 0.809, 0.939) | Timestamp: 08-07-2025 15:31:42
Junta: 6, Angulo: 90.0 | Posicao: (-0.600, 0.809, 0.939) | Timestamp: 08-07-2025 15:32:04
Junta: 5, Angulo: 90.0 | Posicao: (-0.600, 0.798, 1.063) | Timestamp: 08-07-2025 15:32:29

```

Figura 10: Arquivo historiador.txt após série de comandos.



```
2025-07-08 15:23:11 - Px: 0.098, Py: 0.475, Pz: 0.215
2025-07-08 15:23:26 - Px: 0.098, Py: 0.475, Pz: 0.215
2025-07-08 15:23:41 - Px: 0.098, Py: 0.475, Pz: 0.215
2025-07-08 15:23:56 - Px: -1.063, Py: 0.475, Pz: 0.868
2025-07-08 15:24:11 - Px: -1.063, Py: 0.475, Pz: 0.868
2025-07-08 15:24:26 - Px: -1.063, Py: 0.475, Pz: 0.868
2025-07-08 15:24:41 - Px: -0.481, Py: 0.475, Pz: 0.369
2025-07-08 15:24:56 - Px: -0.481, Py: 0.475, Pz: 0.369
2025-07-08 15:25:11 - Px: -0.481, Py: 0.475, Pz: 0.369
2025-07-08 15:25:26 - Px: -0.538, Py: 0.475, Pz: 0.890
2025-07-08 15:29:40 - Px: 0.098, Py: 0.475, Pz: 0.215
2025-07-08 15:29:55 - Px: 0.098, Py: 0.475, Pz: 0.215
2025-07-08 15:30:10 - Px: 0.098, Py: 0.475, Pz: 0.215
2025-07-08 15:30:25 - Px: -1.299, Py: 0.475, Pz: 0.444
2025-07-08 15:30:40 - Px: -1.299, Py: 0.475, Pz: 0.444
2025-07-08 15:30:55 - Px: -1.208, Py: 0.475, Pz: 0.663
2025-07-08 15:31:10 - Px: -1.206, Py: 0.808, Pz: 0.329
2025-07-08 15:31:25 - Px: -0.600, Py: 0.809, Pz: 0.939
2025-07-08 15:31:40 - Px: -0.600, Py: 0.809, Pz: 0.939
2025-07-08 15:31:55 - Px: -0.600, Py: 0.809, Pz: 0.939
2025-07-08 15:32:10 - Px: -0.600, Py: 0.809, Pz: 0.939
2025-07-08 15:32:25 - Px: -0.600, Py: 0.809, Pz: 0.939
2025-07-08 15:32:40 - Px: -0.600, Py: 0.798, Pz: 1.063
2025-07-08 15:32:55 - Px: -0.600, Py: 0.798, Pz: 1.063
2025-07-08 15:33:10 - Px: -0.600, Py: 0.798, Pz: 1.063
2025-07-08 15:33:25 - Px: -0.600, Py: 0.798, Pz: 1.063
2025-07-08 15:33:40 - Px: -0.600, Py: 0.798, Pz: 1.063
2025-07-08 15:33:55 - Px: -0.600, Py: 0.798, Pz: 1.063
2025-07-08 15:34:10 - Px: -0.600, Py: 0.798, Pz: 1.063
2025-07-08 15:34:25 - Px: -0.600, Py: 0.798, Pz: 1.063
2025-07-08 15:34:40 - Px: -0.600, Py: 0.798, Pz: 1.063
2025-07-08 15:34:55 - Px: -0.600, Py: 0.798, Pz: 1.063
2025-07-08 15:35:10 - Px: -0.600, Py: 0.798, Pz: 1.063
2025-07-08 15:35:25 - Px: -0.600, Py: 0.798, Pz: 1.063
2025-07-08 15:35:40 - Px: -0.600, Py: 0.798, Pz: 1.063
```

Figura 11: Arquivo mes.txt após série de comandos.

Analisando as Figuras 10 e 11 conclui-se que os programas estão funcionando conforme o esperado.

6 Conclusão

O trabalho permitiu integrar conceitos de comunicação industrial, controle distribuído e supervisão em ambiente de simulação realista, demonstrando a importância do protocolo OPC UA na Indústria 4.0.